

Desarrollo de una celda reconfigurable de CGRA

Duarte Sanchez David, Jimenez Ulate Eddy, Ruiz Rivera Jeffrey, Esquivel Arias Fabian Zúñiga Ramírez Jose Eduardo

Grupo de 5

{davidduarte28, e.jimenez.10, yr1ruiz, fabiesqui22, je.zuniga}@estudiantec.cr

I. INTRODUCCIÓN

Las arquitecturas reconfigurables han sido muy exitosas en la industria debido a que la evolución del software ha presentado una rápida evolución, con diversas aplicaciones, distintas necesidades de usuario y progreso científico, por lo que el hardware no se puede adaptar en la misma velocidad al software [?]. Sistemas de propósito específico pueden sufrir de obsolescencia rápida y si el precio del desarrollo y producción del hardware es alto, se vuelve inviable económicamente [?].

Actualmente, arquitecturas como las FPGA (*Field Programmable Gate Arrays*) han sido ampliamente adoptadas en diversas aplicaciones debido a su flexibilidad y capacidad de personalización, tomando fuerza en Computación de alto desempeño (HPC) por sus siglas en inglés. Sin embargo, las FPGAs presentan ciertas limitaciones como tiempos de configuración lentos, largos tiempos de compilación y aunque es algo relativo, bajas frecuencias de operación [?].

Debido a lo anterior, se ha tomado un interés en HPC en utilizar arquitecturas como los CGRAs (*Coarse-Grained Reconfigurable Arrays*) que ofrecen una solución intermedia entre los FPGAs y los procesadores de propósito general (GPP). Los CGRAs son arquitecturas de hardware que conceptualmente permiten que el silicio sea maleable y su funcionalidad pueda reconfigurarse de manera dinámica [?]. Los CGRA tienen la ventaja de ser uno o dos ordenes de magnitud más eficientes que las FPGAs y más de 2 a 3 ordenes de magnitud más eficientes que los GPPs [?].

El desarrollo de una celda reconfigurable de CGRA es de gran importancia por su relevancia no solo en HPC, si no también en su adopción en aplicaciones de *Machine-learning*, procesamiento de imágenes y visión por computadora, comunicaciones y otras áreas [?]. Por lo tanto el objetivo de este trabajo es el desarrollo de un CGRA heterogéneo, que posea tentativamente celdas de ruteo de datos, celdas con bancos de memoria distribuida, celdas de procesamiento escalar y celdas de procesamiento vectorial, con la capacidad de ejecutar programas utilizando el set de instrucciones RISC-V, con la capacidad de ejecutar lógica de ocho, 16 y 32 bits.

La importancia de utilizar este set de instrucciones es que este es un estándar abierto, facilitando su adopción debido a que no requiere pagos por licencias. Este es un set de instrucciones basado en la simplicidad, haciendo que el diseño de hardware sea sencillo y una implementación más directa [?].

II. ANTECEDENTES Y TRABAJO RELACIONADO

En los últimos años, las CGRA han ganado un protagonismo significativo como aceleradores de hardware debido a su capacidad para ofrecer un equilibrio excepcional entre la flexibilidad del software y el alto rendimiento del hardware de aplicación específica [?], [?]. A diferencia de las arquitecturas de grano fino, las CGRAs operan a nivel de palabra o bloque, lo que reduce sustancialmente la sobrecarga de reconfiguración y enrutamiento, haciéndolas particularmente atractivas para aplicaciones eficientes en el borde (*edge computing*). El desarrollo e investigación de estas arquitecturas ha sido fuertemente impulsado por la adopción del repertorio de instrucciones (ISA) de código abierto RISC-V [?], cuya naturaleza extensible facilita la creación de Sistemas en Chip (SoC) heterogéneos altamente personalizados.

Investigaciones recientes han demostrado las ventajas de integrar estrechamente núcleos RISC-V con arreglos CGRA para superar el cuello de botella tradicional de Von Neumann. Por ejemplo, [?] exploran la integración de un acelerador CGRA acoplado a un núcleo de alto rendimiento RISC-V CVA6 enfocado en ecosistemas cloud-edge, mientras que [?] proponen DVHetero, un entorno de trabajo integral para el diseño y la validación de SoCs heterogéneos basados en esta combinación.

A nivel de implementación práctica, estas arquitecturas se materializan en esfuerzos de código abierto de vanguardia; repositorios como *risc-v-cgra* (desarrollado por ECASLab) investigan estas integraciones a nivel de sistema, mientras que proyectos como OpenEdgeCGRA (ESL-EPFL) proporcionan plataformas open-hardware completas. En este último, la arquitectura se compone de una malla bidimensional configurable de Elementos de Procesamiento (PEs) o celdas reconfigurables que interactúan en forma de toroide y comparten registros para maximizar la paralelización de instrucciones con una sobrecarga de memoria mínima. Otro proyecto destacable como OpenCGRA desarrollado por el PNNL (*Pacific Northwest National Laboratory*) el cual genera procesadores reconfigurables de forma parametrizable, de esta manera, permite diseñar el hardware por código como el tamaño de la malla, el tipo de celdas, cómo se comunican entre ellas, luego se prueba el funcionamiento usando Python y Verilator y finalmente se exporta el diseño a un código de diseño de hardware estándar (HDL) como Verilog.

Para que este tipo de sistemas alcance su máximo potencial, el diseño interno de la celda reconfigurable individual (PE) y

las herramientas de compilación asociadas son críticos [?]. Destacan la necesidad de un mapeo consciente del contexto de memoria (context-memory aware mapping) para asegurar una aceleración eficiente desde el punto de vista energético, subrayando que la latencia en el acceso a datos dicta el rendimiento global. En este marco, el diseño y desarrollo de una nueva celda reconfigurable de CGRA se fundamenta en la necesidad de optimizar las interconexiones locales, el acceso determinista a operandos intermedios y la versatilidad de sus operaciones a nivel de ciclo de reloj, retos que continúan siendo áreas de activa exploración en la frontera del diseño de aceleradores heterogéneos.

III. ANÁLISIS DE SOLUCIONES EXISTENTES (SOTA)

El diseño de soluciones óptimas requiere un escrutinio riguroso de las alternativas vigentes en la literatura científica. En esta sección se evalúan las fortalezas y debilidades de los enfoques predominantes en el estado del arte (SOTA), delimitando sus escenarios de viabilidad técnica.

III-A. Estudio Comparativo de Propuestas Vigentes

Las soluciones actuales presentan compromisos fundamentales (*trade-offs*) entre flexibilidad, eficiencia y costo de implementación:

- **Enfoque Basado en Procesadores de Propósito General (GPP):** Destaca por su alta flexibilidad de programación y madurez en cadenas de desarrollo. No obstante, presenta debilidades críticas en entornos de tiempo real debido al determinismo limitado y un consumo energético ineficiente bajo cargas masivas de datos. Es viable únicamente en escenarios donde las restricciones de potencia y latencia son secundarias.
- **Enfoque Basado en Aceleradores Dedicados (ASIC):** Ofrece el máximo rendimiento por unidad de área y un consumo energético mínimo. Sin embargo, su rigidez absoluta impide la reconfiguración ante actualizaciones algorítmicas, además de acarrear costos de desarrollo inviables para producciones de bajo volumen.
- **Enfoque Basado en Lógica Programable (FPGAs / Arquitecturas Reconfigurables):** Representa un punto medio balanceado. Permite la optimización a nivel de bits y paralelismo masivo. Su principal desventaja radica en la alta complejidad de diseño, tiempos de compilación elevados y la gestión compleja de la memoria compartida.

Para ilustrar de forma concisa estos compromisos, la Tabla I resume los criterios de viabilidad tecnológica encontrados en la literatura.

III-B. Lista de Desafíos Observados

A partir del análisis sistemático del SOTA, se derivan los siguientes desafíos técnicos abiertos que justifican el desarrollo de una nueva alternativa:

- D1. Rigidez Estructural:** Incapacidad de los aceleradores actuales para adaptarse a variaciones dinámicas en el dominio de los datos sin comprometer severamente el *timing*.

Cuadro I
MATRIZ COMPARATIVA DE SOLUCIONES EN EL ESTADO DEL ARTE

Solución	Fortalezas	Debilidades	Viabilidad
Propuesta A [?]	Baja latencia, optimización espacial.	Rigidez de hardware, alto tiempo de diseño.	Dominios fijos estables.
Propuesta B [?]	Alta flexibilidad, reconfiguración ágil.	Alto consumo de recursos, <i>overhead</i> .	Entornos dinámicos no c
Propuesta C [?]	Procesamiento masivo por ciclo (OP/s).	Consumo energético restrictivo.	Infraestructura fija/servid

D2. Eficiencia de Recursos Embebidos: Degradación del rendimiento de procesamiento por unidad de área o celda cuando se escalan los tamaños de datos.

D3. Determinismo Temporal: Cuellos de botella en la latencia de interconexión que impiden garantizar restricciones de tiempo real estricto.

IV. PROPUESTA DE SOLUCIÓN Y ARQUITECTURA DE II NIVEL

Para mitigar las deficiencias identificadas en la Sección III, se propone una arquitectura de segundo nivel orientada a optimizar el flujo de datos y la reconfigurabilidad del sistema.

IV-A. Descripción General de la Solución y Alcance

La solución consiste en un enfoque novedoso que introduce abstracción y modularidad en la lógica de procesamiento. A diferencia de las soluciones monolíticas del SOTA, esta arquitectura desacopla las unidades funcionales de la red de interconexión dinámica. El alcance de esta propuesta abarca el modelo formal de la arquitectura, su diseño a nivel de bloques lógicos interconectados, y su posterior validación mediante modelos de simulación ciclo a ciclo. Queda fuera del alcance actual la fabricación física del silicio o la implementación en silicio comercial de escala nanométrica.

IV-B. Arquitectura de Segundo Nivel (Detalle de Componentes)

La organización jerárquica de la solución propuesta se detalla en la Fig. 1.

Figura 1. Diagrama de bloques de la arquitectura de nivel II propuesta. Se detallan las interconexiones entre los bloques de control, celdas de procesamiento jerárquico y la interfaz de memoria compartida.

La arquitectura de nivel II segmenta las responsabilidades en tres capas fundamentales:

- **Capa de Control y Secuenciación:** Gestiona el *timing* general y despacha las señales de configuración a las unidades lógicas de forma determinista.
- **Matriz de Procesamiento Reconfigurable:** Unidades de procesamiento optimizadas para ejecutar operaciones masivas (OP/s) en paralelo.
- **Subsistema de Memoria de Acceso Local:** Minimiza la latencia de acceso a datos mediante buffers balanceados que mitigan los desafíos de ancho de banda.

IV-C. Justificación frente a los Desafíos del SOTA

La novedad y viabilidad de esta arquitectura se fundamenta en cómo resuelve individualmente los desafíos planteados:

- **Solución a D1:** La inclusión de lógica reconfigurable de grano grueso permite adaptar las celdas funcionales a diferentes dominios de datos sin reconstruir la infraestructura de hardware.
- **Solución a D2:** Mediante el particionamiento jerárquico de segundo nivel, el consumo de recursos escala de forma lineal ($O(N)$) y no exponencial respecto al tamaño de los datos.
- **Solución a D3:** Se garantiza el cumplimiento estricto del *timing* mediante una red de interconexión segmentada (*pipelined*) libre de colisiones latentes.

V. PLAN DE EXPERIMENTACIÓN Y METODOLOGÍA

La validación cuantitativa de la arquitectura propuesta frente a los enfoques existentes se regirá por un plan experimental estructurado y replicable.

V-A. Casos de Prueba y Variabilidad de Datos

Para comprobar la robustez y escalabilidad de la propuesta, los experimentos se dividirán según dos ejes de variabilidad:

1. **Variación en el Tamaño de Datos:** Pruebas de carga incremental utilizando matrices y vectores de datos de dimensiones pequeñas (16×16), medianas (128×128) y grandes (1024×1024).
2. **Diversidad en el Dominio de Datos:** Evaluación bajo conjuntos de datos con perfiles de dispersión y patrones de acceso aleatorios, estructurados y secuenciales dispersos, simulando comportamientos de diferentes aplicaciones reales.

V-B. Métricas de Evaluación Técnica

El contraste con el estado del arte se cuantificará mediante las siguientes métricas de ingeniería de hardware y sistemas:

- **Consumo de Recursos:** Porcentaje de ocupación de elementos lógicos, registros y bloques de memoria en el entorno de evaluación.
- **Latencia:** Número exacto de ciclos de reloj requeridos desde la entrada del estímulo hasta la estabilización de la respuesta de salida.
- **Timing (Frecuencia Máxima):** Análisis de restricciones temporales enfocado en el peor camino crítico (*Worst Negative Slack*, WNS) para derivar la frecuencia máxima de operación (f_{max}).

- **Rendimiento de Carga (OP/s):** Cantidad de operaciones efectivas ejecutadas por segundo (Giga-Operaciones por Segundo, GOPS) bajo procesamiento continuo.

V-C. Herramientas y Entorno de Desarrollo

Con el fin de asegurar la reproducibilidad estricta de las pruebas de diseño y simulación, se define la infraestructura de herramientas de software con sus respectivas versiones en la Tabla II.

Cuadro II
HERRAMIENTAS DE DESARROLLO Y VERSIONES DEL ENTORNO

Herramienta / Software	Versión	Propósito en el Experimento
MATLAB / Simulink	R2026a	Modelado matemático y <i>golden model</i> .
Xilinx Vivado Design Suite	2025.2	Síntesis lógica y análisis de <i>timing</i> .
ModelSim / Questa Advanced	2025.1	Simulación funcional RTL ciclo a ciclo.
Docker Engine (Ubuntu Base)	24.0.9	Contenerización del flujo de compilación.

DECLARACIÓN DE USO DE HERRAMIENTAS DE IA

Para este trabajo se hizo uso de Inteligencia Artificial para la ayuda en la estructura de la introducción y el estado del arte, así como para una revisión gramatical y ortográfica del documento.